

Thesis Setup — Full Explanation

Deploying knowledge graphs with QLever, computing all 13 thesis metrics in Rust + SPARQL, and a niceGUI dashboard

Author: Dharmang Pambhar | Updated: 19 August 2026 | Advisor: Samuel García (TUM)

This document explains, from the ground up, what the thesis task is, the tools involved, everything that was set up on the machine, how the pieces fit together, and what remains. It is meant to be readable on its own and useful to bring to an advisor meeting.

1. What this project is about

The thesis goal is to **compute metrics over a knowledge graph** (YAGO or DBpedia) and present them in a dashboard. A *knowledge graph* is a large set of facts written as **triples** — *subject – predicate – object*, e.g. Einstein – bornIn – Ulm. The pipeline has four layers:

```
KNOWLEDGE GRAPH DATA      YAGO / DBpedia (RDF triples)
| loaded into
v
QLever = the database / RDF store
- 'index' once -> builds fast lookup structures on disk
- 'start'      -> serves a SPARQL endpoint (a server on a port)
| queried via SPARQL over HTTP
v
YOUR METRICS CODE (Rust + SPARQL, like Knowgly)
- SPARQL counts -> Rust HashMaps -> compute metrics
| display
v
niceGUI = the dashboard (Python -> charts, tables in the browser)
```

The four tools map cleanly onto these layers:

Tool	Role	Analogy
YAGO / DBpedia	The data (a knowledge graph of facts)	the raw material
QLever	The database that stores & queries it via SPARQL	like Postgres, but for graphs
Knowgly	A reference metrics project (to understand, not copy)	a worked example
niceGUI	The dashboard to display your metrics	the front-end

2. What QLever actually is

- **Triples & SPARQL**: RDF data is triples; **SPARQL** is its query language (like SQL is for tables).
- **QLever** is a very fast SPARQL engine from the University of Freiburg.
- **qllever index** reads the raw triple files *once* and builds an optimized index on disk (the slow, one-time step).
- **qllever start** launches a server that answers SPARQL queries over HTTP — so you can query it from Python.
- A **Qleverfile** is the recipe/config: which dataset, how to index it, which port, etc.
- The `qllever` command you install via pip is only the **controller**; the heavy engine runs separately (here, inside a container).

3. The machine and its constraints

Knowing the hardware explains several decisions made during setup.

Property	Value	Why it mattered
Computer	MacBook Air, Apple Silicon (arm64), 16 GB RAM	Limited RAM -> can't index 1.3B triples from scratch comfortably
Disk free	~140 GB at start	Enough for the 24 GB zip + 41.6 GB index
Docker	Not installed; no Homebrew; no admin sudo in tooling	Had to install a Docker engine without a password
Python	System Python 3.9 (too old)	qllever needs Python 3.12+

4. Everything that was set up (in order)

Step	What & why
1. uv + Python 3.12	Installed 'uv' (user-space Python manager) and created a Python 3.12 virtual environment, because the system Python 3.9 was too old for qllever.
2. qllever CLI	Installed the qllever control tool into the venv (uv pip install qllever).
3. Qleverfiles	Downloaded the YAGO, DBpedia and olympics recipes from the qllever-control repo.
4. Pre-built YAGO index	Downloaded the advisor's ready-made YAGO 4.5.0.2 index (skips the hours-long 'index' step).
5. Verified the download	First download was truncated by ~1 GB; proved it via Zip64 analysis and re-downloaded the full 23.88 GB, then extracted 41.6 GB.
6. Docker engine (Colima)	Installed Colima + Lima + Docker CLI into ~/.local/bin with NO admin password, and booted a lightweight Linux VM.
7. qllever start	Served the index at localhost:9004 using a QLever image pinned to the exact build that created the index.
8. Python querying	Wrote sparql.py + metrics_examples.py and confirmed real results against the local index.
9. Web UI	Launched the QLever UI at localhost:8176 and pointed it at the local index.
10. Moved off iCloud	Discovered the Desktop is iCloud-synced (it corrupted the venv); moved the whole project to ~/thesis (not synced).
11. First Rust + SPARQL metric	Built rust_metrics/ (Entity Type Importance), writing results to JSON + CSV.
12. niceGUI dashboard	Built dashboard.py — reads the metric's JSON and charts each type's characteristic predicates at localhost:8080.

5. The YAGO index (the data)

The advisor shared a finished QLever index, so the slow 'qllever index' step was skipped. It is **YAGO version 4.5.0.2**. Key facts read from its metadata:

Property	Value
Triples (normal)	1,305,431,407 (~1.3 billion)
Distinct subjects (entities)	49,688,174
Distinct objects	645,362,721
Distinct predicates	123
On-disk size	~41.6 GB (28 files)
Index build commit	e5f672 (Oct 2024)

Vocabulary	on-disk compressed dictionary (IRIs <-> integer IDs)
------------	--

An index is made of **six permutations** (the triples sorted in every order: SPO, SOP, PSO, POS, OSP, OPS) so QLever can look up data quickly in any direction, plus the **vocabulary** (the dictionary that maps every IRI/string to an integer ID).

6. The query engine (Colima / Docker)

QLever's engine is a Linux C++ server with **no native macOS binary**, so it runs in a container. Docker was installed **without any admin password** using user-space binaries:

- **Colima + Lima + Docker CLI** placed in `~/local/bin` (no Homebrew, no sudo).
- Colima boots a small Linux VM using macOS's built-in Virtualization framework (vz).
- Your `~/thesis` folder is shared into the VM (virtiofs) so the container can read the 41.6 GB index.
- The QLever image is **pinned** to `commit-e5f6724` — the exact build that made the index — so the index format is guaranteed compatible (`'latest'` could reject a 2024-format index).

Endpoint: **http://localhost:9004** | Access token: `yago-4.5.0.2`

Note: visiting `http://localhost:9004` in a browser shows "Unknown path". That is normal — it is a SPARQL API, not a web page; it answers queries, not page requests.

7. The project files and folders

```
~/thesis/
  qlever-workspace/      <- project root
    rust_metrics/        <- THESIS METRICS (Rust + SPARQL, mirrors Knowgly)
      Cargo.toml
      src/qlever_client.rs <- SPARQL client (can hold TWO endpoints at once)
      src/common.rs       <- shared queries + entropy helpers
      src/out.rs          <- writes results/[<label>]/<metric>.{json,csv}
      src/<metric>.rs      <- ONE MODULE PER METRIC (13 + 2 baselines)
      src/main.rs         <- subcommand dispatch ('help' lists everything)
      results/[<label>]/*.json,csv <- metric output, one folder per snapshot
  dashboard.py           <- niceGUI dashboard (reads results JSON, charts at :8080)
  sparql.py              <- lightweight Python query reference
  metrics_examples.py    <- Python example metrics (reference)
  make_pdf.py            <- generates this PDF
  RUNBOOK.md             <- start/stop/query/metrics commands
  Qleverfiles/           <- recipes (yago-4, dbpedia, olympics)
  yago/                  <- THE DATABASE (41.6 GB index, not code)
  .venv/                 <- isolated Python 3.12 + qlever + requests
  Thesis_Setup_Explained.pdf <- this document
```

Categories: **thesis metrics** (the Rust project `rust_metrics/`), the **dashboard** (`dashboard.py`), **config** (Qleverfiles), **data + tooling** (`yago/`, `.venv/` — set up once), and a small **Python reference** (`sparql.py`, `metrics_examples.py`).

8. The metric implementation (Rust + SPARQL)

Per the advisor and the Knowgly reference, the metrics are implemented as a **mix of SPARQL and Rust working with dictionaries (HashMaps)**: SPARQL does the heavy counting on the server, and Rust holds the aggregated counts and applies the formula. The project is `rust_metrics/` and mirrors Knowgly's structure. **All 13 metrics of the final metrics list are implemented** — about 2350 lines across 19 modules, one module per metric plus a subcommand.

File	Role
<code>src/qlever_client.rs</code>	SPARQL client. Holds a global endpoint for the per-snapshot metrics and can build a second one for the cross-version metrics. Surfaces QLever's own error text, and aborts on a failed query rather than returning an empty result.
<code>src/common.rs</code>	Shared queries (top types, <code> E_t </code> , class and predicate lists) and the entropy helpers.
<code>src/out.rs</code>	Result writing: <code>results/[<label>]/<metric>.json + .csv</code> .

src/<metric>.rs	One module per metric — the formula and its queries.
src/main.rs	Subcommand dispatch, environment handling, and the 'all' runner.

The 13 metrics (baselines etimp and pagerank come from Knowgly and are not thesis contributions):

#	Metric	Command	Level
1	Class population & share	population	class
2	Property entropy per type	entf	class
3	Entropy-weighted type importance	entetimp	class
4	Class entropy	classentropy	class
5	Entity informativeness	inforank	entity
6	Object diversity	diversity	entity
7	Entropy-weighted PageRank (own variant)	entropy-pagerank	global
8	Graph size & shape	shape	global
9	Class-level change rate (churn)	churn	global, 2 endpoints
10	Triple diff, integer-encoded	diff	global, 2 endpoints
11	Metric trajectories	trajectories	global, stored results
12	Vocabulary / schema evolution	vocab	global, 2 endpoints
13	Cross-KG class comparison	crosskg	global, 2 endpoints

How to run them

```
source "$HOME/.cargo/env"
cd ~/thesis/qlever-workspace/rust_metrics
export QLEVER_ENDPOINT=http://localhost:9004
export QLEVER_LABEL=yago-4.5.0.2          # names this snapshot -> results/<label>/
cargo run --release all 8                 # every single-endpoint metric, top-8 classes
cargo run --release help                  # the full list

# comparing two versions needs a second endpoint
export QLEVER_ENDPOINT_B=http://localhost:9005
cargo run --release diff 'http://schema.org/Person' 5000
cargo run --release trajectories graph_shape yago-4 yago-4.5.0.2
```

The QLEVER_LABEL is what makes evolution work: the same metric run against two versions under two labels lands in two folders, and metric 11 reads both back and reports the deltas.

Three implementation decisions worth knowing

1. Entropy is computed from a count-of-counts histogram. Shannon entropy depends only on the multiset of counts, never on the values themselves, so there is no reason to stream millions of object values over HTTP. A nested GROUP BY makes QLever fold them into a histogram of a few hundred rows, and the entropy is recovered exactly:

```
H = log2 N - (1/N) * sum_c m_c * c * log2 c
m_c = how many distinct values occur exactly c times
N   = total value occurrences          (exact, not sampled)
```

2. The triple diff uses a subject window. The obvious way to take a comparable slice of two versions is ORDER BY ?s ?p ?o LIMIT n, but that forces the server to sort the entire graph before it can take n rows — on a 1.5 billion triple graph QLever asked for 12.7 GB and refused. Instead the diff takes the lexicographically first n subjects of each version (ordering a subject list is an index scan, not a sort of every triple), cuts both lists at the smaller upper bound, and fetches those subjects' triples by name. Inside that window both versions are

covered completely, so the diff is exact there — including entities that appear or vanish — and nothing needs a global sort. Metrics 9 and 10 should therefore always be scoped to a class.

3. Metric 10 compares two versions of one KG, not two different KGs. YAGO and DBpedia use disjoint subject IRIs, so the window falls entirely inside one of them and every triple of the other is reported as 'added' — arithmetically correct and completely meaningless. The code detects this and says so. Comparing two different KGs is metric 13's job, which matches classes by name and compares their metric values instead.

The efficiency contribution, measured

Metric 10 encodes every term into a u32 so a triple becomes 12 bytes and the comparison is a sort plus a linear merge over integers, instead of hashing three strings per triple. Both the integer path and the naive string path are implemented and timed against each other, and the code asserts that the two agree — so the benchmark cannot drift away from being correct. Measured on real data:

Path	Time	Memory
integer-encoded sets	43 ms	3.3 MB
naive string hash sets	259 ms	53.9 MB
ratio	6.0x faster	16.5x smaller

Verification

All 13 metrics were run against live endpoints. Metrics 1-8 on the olympics dataset and on YAGO; metrics 9 and 10 self-tested against a single endpoint, where the diff must be exactly zero, and then at scale on YAGO; metric 12 across two different graphs; metric 13 on YAGO versus DBpedia (Person: 6.43 M entities at 20.6 bits of class entropy against DBpedia's 1.86 M at 17.4 bits); metric 11 across two stored snapshots.

Metric 7 also demonstrates its own premise on the olympics graph: the predicate weights come out as athlete = 1.000 against type = 0.008, so the low-entropy class nodes drop out of the top ranks and every athlete rises exactly five places against the unweighted baseline.

Python files (reference only). `sparql.py` and `metrics_examples.py` are a small Python query helper kept for quick checks — they are *not* the thesis metric implementation (that is the Rust project above).

9. The dashboard (niceGUI)

The final layer visualises the metric. `dashboard.py` is a **niceGUI** web app (Python) that reads the Rust metric's output (`rust_metrics/results/entity_type_importance.json`) and presents it interactively in the browser — no SPARQL is run at view time, it simply reads the pre-computed JSON, so the page is instant.

What it shows:

- A **dropdown** to pick any entity type that the metric was computed for (Person, Galaxy, Politician, ...).
- A **Top-N selector** (3–40) controlling how many predicates are charted.
- An interactive **horizontal bar chart** (ECharts) of that type's most characteristic predicates, ranked by ETImp score, with the value labelled on each bar.
- A **full ranked table** of every predicate for the type (rank, predicate, ETImp score), paginated and sortable.

How it is built: long IRIs are trimmed to their readable last segment (e.g. `...#birthDate` -> `birthDate`); the chart and table are wrapped in an `@ui.refreshable` block so changing the type or Top-N redraws them without reloading the page. The JSON is read once at startup, so to pick up regenerated data you restart the app.

Run it (after the metric has produced results/):

```
cd ~/thesis/qllever-workspace
source .venv/bin/activate
python dashboard.py          # then open http://localhost:8080
```

This completes the full pipeline end to end: YAGO index -> QLever (:9004) -> Rust + SPARQL metric -> results JSON -> niceGUI dashboard (:8080).

10. How to start / stop / query (runbook)

Set up the shell each session:

```
export PATH="$HOME/.local/bin:$PATH"
source ~/thesis/qlever-workspace/.venv/bin/activate
```

Start everything (e.g. after a reboot):

```
colima start
cd ~/thesis/qlever-workspace/yago
qlever --qleverfile Qleverfile start      # serves localhost:9004
```

Check / stop:

```
docker ps          # see the running server
qlever --qleverfile Qleverfile stop      # stop QLever
colima stop         # stop the Docker VM (frees RAM)
```

Query from the browser UI: **<http://localhost:8176/yago-4>**

11. The Knowgly metrics — the idea

The advisor asked you to **understand** Knowgly's metrics (not reimplement them). The whole approach is one pattern:

Metric = SPARQL does the counting (GROUP BY ... COUNT) -> code plugs the counts into a formula -> results stored in nested dictionaries (Type -> Predicate -> value). The "Rust for dictionaries" the advisor mentioned is just bookkeeping of those counts; in Python it is a plain dict.

Knowgly implements two metrics that rank how important each predicate is for each entity type:

- **Entity Type Importance** (a TF-IDF for predicates): $ETImp(p,t) = EF_p(p,t) \times \log_2(|E_t| / EF_p(p,t))$, where $EF_p(p,t)$ is the number of entities of type t that use predicate p , and $|E_t|$ is the total entities of type t . Predicates used by every entity of a type score 0 (uninformative); characteristic predicates score high.
- **Entropy Type Importance** (Shannon entropy of a predicate's object values): $H = -\sum P(o) \log P(o)$. High entropy = diverse, informative values (name, birthDate); low entropy = near-constant values.

Engineering lessons worth reusing even for simple metrics:

- Do NOT write one giant query; split into many small per-type queries (avoids query-planner blowups).
- Run them in parallel.
- Aggregate the counts in dictionaries.
- Optionally cluster predicates by their scores into importance groups.

Your thesis metrics are **simpler than Knowgly's** but implemented the **same way — SPARQL + Rust dictionaries**. The first one, Entity Type Importance, is already built in `rust_metrics/` (see Section 8) and runs against your local YAGO.

12. Does this cover the supervisor's instructions?

Supervisor's instruction	Status
Explore / use QLever to deploy YAGO locally	DONE
Install qlever via pip	DONE (in a Python 3.12 venv)
Grab a Qleverfile	DONE (yago / dbpedia / olympics)
qlever ... index	SKIPPED ON PURPOSE - used the pre-built index instead
Use the pre-built YAGO index (SharePoint)	DONE (downloaded, verified, extracted)
qlever ... start	DONE (serving localhost:9004)
Query as a SPARQL endpoint with Python	DONE (sparql.py + metrics_examples.py)

Everything in the message is implemented. The only line not literally executed is `qlever index` — intentionally, because the advisor's pre-built index exists precisely so you can skip that slow step.

13. Current status and next steps

Thesis piece	Status
Deploy a KG locally with QLever	DONE
Query SPARQL (Python + Rust)	DONE
Understand the Knowgly metrics	DONE
Final metrics list agreed (13 metrics, 3 levels)	DONE

All 13 metrics implemented in Rust + SPARQL	DONE
Metrics verified against live endpoints	DONE
Efficiency benchmark for the triple diff	DONE (6.0x faster, 16.5x smaller)
Results written as JSON + CSV, one folder per snapshot	DONE
niceGUI dashboard (charts ETImp + PageRank at :8080)	PARTIAL
Dashboard reads the other 11 metrics + results/<label>/	TODO
Index a second YAGO version and run the cross-version metrics on it	TODO
Add DBpedia versions	TODO

The computation side of the thesis is complete: every metric on the final list is implemented, runs against a real endpoint, and writes its results. Two things remain. First, the dashboard still only knows about the two original result files, so it needs to learn the other eleven metrics and the per-snapshot folder layout. Second, the cross-version metrics (9, 10, 11, 12) have been verified for correctness but not yet run on two real versions of YAGO — that needs the second index finished on the server, at which point they produce thesis results rather than test results.